

Reviewer Pack — Minimal Index of Local Indeterminacy in Category Theory and ...

Entry dba8f730a86a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 19:19:38 UTC

Minimal Index of Local Indeterminacy in Category Theory and Resolution Proof Width

Entry ID: dba8f730a86a **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 19:19:38 UTC

1. Conjecture

Field A (mathematical branch): Category Theory **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every Tseitin formula φ with n variables, the minimal index of local indeterminacy ($i\varphi$) of its associated category $C(\varphi)$ is linearly correlated with its resolution proof width $w(\varphi)$, such that $i\varphi = \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

Category theory provides a framework for understanding complex structures in abstract terms. Local indeterminacy measures the complexity of these structures, which might correlate with the difficulty of proving properties about them, like resolution proof width. If such a correlation holds, it could lead to new insights into the nature of proof complexity.

Taxonomy category: Category_Theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 05ccfd12d7da7c49

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For Tseitin formulas φ with n variables, if the correlation coefficient between minimal index of local indeterminacy ($i\varphi$) and resolution proof width ($w(\varphi)$) is ≥ 0.8 over 30 random seeds, then support for the conjecture is provided.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "category theory" AND "resolution proof complexity" AND minimal index of local indeterminacy" - " $i\phi = \theta(w(\phi))$ " site:arxiv.org" - "Tseitin formula" AND resolution width AND category C(ϕ)

Top relevant hits considered: - [http://arxiv.org/abs/2103.09609v1] Characterizing Tseitin-formulas with short regular resolution refutations - [http://arxiv.org/abs/2402.00542v1] Hardness of Random Reordered Encodings of Parity for Resolution and CDCL - [http://arxiv.org/abs/1004.2159v2] Algebraic Proofs over Noncommutative Formulas

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        factor = Fraction(matrix[i][i])
        for j in range(i, n + 1):
            matrix[i][j] /= factor
        for j in range(n):
            if i != j:
                factor = Fraction(matrix[j][i])
                for k in range(i, n + 1):
                    matrix[j][k] -= factor * matrix[i][k]
    return matrix

def determinant(matrix):
    n = len(matrix)
    det = Fraction(1)
    for i in range(n):
        det *= matrix[i][i]
    return det
```

```

def tseitin_formula(n):
    variables = [f'x{i+1}' for i in range(n)]
    clauses = []
    for i in range(n):
        y = f'y{i+1}'
        clauses.append([variables[i], -y])
        clauses.append([-variables[i], y])
        for j in range(i+1, n):
            z = f'z{(i+1)*(n-i)//2+j-i-1}'
            clauses.append([y, z])
            clauses.append([-y, -z])
            clauses.append([z, variables[j]])
            clauses.append([-z, -variables[j]])
    return variables, clauses

def resolution_width(clauses):
    n = len(clauses)
    max_clauses = 0
    for i in range(n):
        for j in range(i+1, n):
            new_clauses = []
            for clause_i in clauses[i]:
                if -clause_i in clauses[j]:
                    continue
            for clause_j in clauses[j]:
                if -clause_j in clauses[i]:
                    continue
            new_clause = list(set(clause_i + clause_j))
            if len(new_clause) == 1:
                return 1
            new_clauses.append(new_clause)
        max_clauses = max(max_clauses, len(new_clauses))
    return max_clauses

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        variables, clauses = tseitin_formula(n)
        i_phi = len(clauses) # Simplified local indeterminacy measure
        w_phi = resolution_width(clauses)

        if i_phi == 0 or w_phi == 0:
            continue

        results.append({
            "n": n,
            "i_phi": i_phi,
            "w_phi": w_phi
        })

    if not results:
        return {
            "metric_name": "resolution_width",

```

```

        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

instances_tested = len(results)
n_max = max(result["n"] for result in results)
i_phi_values = [result["i_phi"] for result in results]
w_phi_values = [result["w_phi"] for result in results]

mean_i_phi = sum(i_phi_values) / instances_tested
mean_w_phi = sum(w_phi_values) / instances_tested

correlation_coefficient = 0
if len(results) > 1:
    numerator = sum((i_phi - mean_i_phi) * (w_phi - mean_w_phi) for i_phi, w_phi in zip(i_phi_values, w_phi_values))
    denominator = math.sqrt(sum((i_phi - mean_i_phi)**2 for i_phi in i_phi_values)) * math.sqrt(sum((w_phi - mean_w_phi)**2 for w_phi in w_phi_values))
    correlation_coefficient = numerator / denominator

conjecture_holds = correlation_coefficient >= 0.8
counterexample = "" if conjecture_holds else "correlation_coefficient < 0.8"

return {
    "metric_name": "resolution_width",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(seed) for seed in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all("conjecture_holds" in r and r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_dev = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
        support_fraction = 1.0
    else:
        mean_value = None
        std_dev = None
        support_fraction = sum(1 for r in results if "conjecture_holds" in r and r["conjecture_holds"]) / len(results)

    if all("conjecture_holds" in r and r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif any("counterexample" in r and r["counterexample"] for r in results):
        counterexample = next(r["counterexample"] for r in results if "counterexample" in r)

```

```

        first_failing_seed = seeds[next(i for i, r in enumerate(results) if "counterexample" in r)]
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6105ee08.py", line 138, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6105ee08.py", line 83, in run_trial
    variables, clauses = tseitin_formula(n)
                          ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6105ee08.py", line 48, in tseitin_formula
    clauses.append([variables[i], -y])
                    ^^^
TypeError: bad operand type for unary -: 'str'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the correlation coefficient could not be calculated to verify the conjecture. | next: Debug the test code to ensure it runs successfully and produces the required data for calculating the correlation coefficient.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18292
2	propose	ollama_remote	glm4:latest	0	0	13312
3	preregistration	ollama_remote	glm4:latest	0	0	16625
4	novelty	ollama_remote	glm4:latest	0	0	11091
5	novelty	ollama_remote	glm4:latest	0	0	17600
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26903
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11092
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11605

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25106
10	judge	ollama_remote	glm4:latest	0	0	8633

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 160258 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/dba8f730a86a.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/dba8f730a86a.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/dba8f730a86a.tar.gz (if generated)